n = 83: closure + index lift replication with repeated alternating pairs (Tier 1)

2026-10-20

n = 83: closure + index lift replication with repeated alternating pairs (Tier 1)

Status (2026-10-20): frozen and launched; no result yet. The protocol (sha256 `bd016acb...`) and the schedule (sha256 `9255542518...`) were committed before any Tier 1 timing. The run is the workflow `.github/workflows/n83-closure-indexlift-rep.yml` on branch `claude/n83-closure-indexlift-rep`. This README states the design only. Every number below comes from the frozen protocol, the parent's archived run (the power prior) or the cloud-agent VM (a local correctness diagnostic, not timing evidence).

Why this run

PR #997 (merged as `e55510070`) made the closure + index-lift M38I producer the n = 83 a = 1 baseline row: end-to-end cold **7.6558 [7.2192, 8.1103]** over the index-lift row, run 39335956567, `koblitz-n83-closure-indexlift-20261019`. That primary had a single cM38I and a single cPI cold observation per point (20 points), plus separate rank blocks (4 kPI, 16 kM38I). Neither the cold arms nor the rank arms were *repeated alternating primary pairs*.

The Tier 1 rule in AGENTS.md ("CPU performance isolation gate" / "CPU performance evidence tiers") requires repeated alternating pairs and a 95% paired confidence interval whose lower bound exceeds 1 before a result is called a Tier 1 speedup. docs/ic/BASELINES.md ("The rule this file serves") asks for a *verified, measured improvement ... under a protocol frozen before timing.*

The owner's qualification notes for PR #986 (`reviews/pr-986-knownlog-qualification-20261020/QUALIFICATION.md` on `claude/n83-knownlog`) and for PR #988 (`reviews/pr-988-onlineframe-qualification-20261019/QUALIFICATION.md` on `claude/n83-onlineframe`) explain this gap: "one ... primary observation per point" plus separate rank or A/A blocks "are not repeated alternating primary pairs". The same applies to #997. This experiment replicates #997 under that rule. It does not change or reinterpret the parent's record.

The producer: unchanged

There is no code change. The parent's derivation chain is copied byte for byte:

1. the rank-probe producer (`d4394df3...`, PR #980);
2. the index-lift producer (`7df72603...`, PR #981);
3. the closure producer (`8916e034...`);
4. the composition `make_online_source.py`.

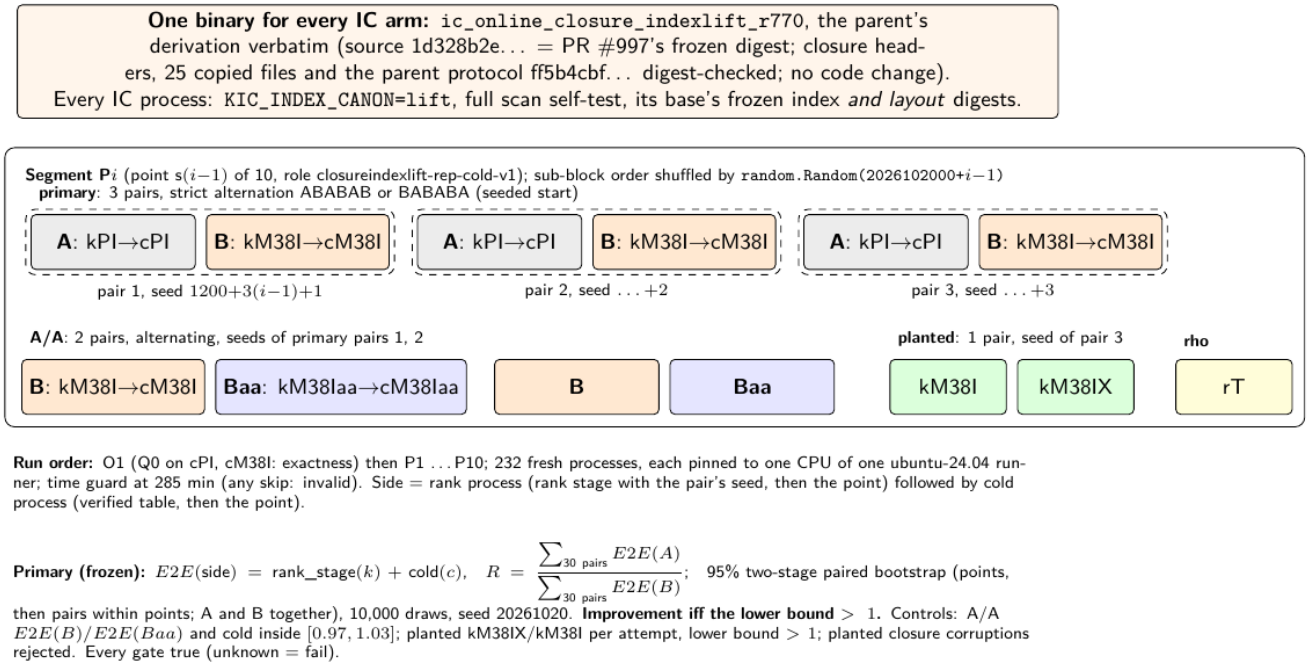
The parent's `arm.py`, `check_log_table.py`, `verify_closure.py`, `make_closure_base.py`, the `online/` and `lift2/` crates and every closure input are also copied byte for byte.

`restore.py` checks every parent digest on every build, and the workflow runs it before the build. It checks:

- the parent protocol's own SHA-256 (`ff5b4cbf...`, pinned in `restore.py`);
- the derived producer, which must equal the parent's frozen `online_source_sha256`, `1d328b2e461a...`;
- both regenerated closure headers (M38 `42e0de3d...`, M10 `ba99464a...`), which must equal the parent's;
- each of the 25 copied files, which must have the digest the parent protocol froze for it (`VERBATIM` in `restore.py`).

On the cloud-agent VM, `restore.py --crypto-repo` (crypto 770f6a69, read only) passed all of these, and `make_online_source.py--check` passed. The bases, the arm configurations and the per-process gates in `protocol.json` are the parent's, copied verbatim (`replication_source` on each).

Frozen design



Run order: O1 (Q0 on cPI, cM38I: exactness) then P1 ... P10; 232 fresh processes, each pinned to one CPU of one ubuntu-24.04 runner; time guard at 285 min (any skip: invalid). Side = rank process (rank stage with the pair's seed, then the point) followed by cold process (verified table, then the point).

Primary (frozen): $E2E(\text{side}) = \text{rank_stage}(k) + \text{cold}(c)$, $R = \dfrac{\sum_{30 \text{ pairs}} E2E(A)}{\sum_{30 \text{ pairs}} E2E(B)}$; 95% two-stage paired bootstrap (points, then pairs within points; A and B together), 10,000 draws, seed 20261020. **Improvement iff the lower bound** > 1. Controls: A/A $E2E(B)/E2E(Baa)$ and cold inside $[0.97, 1.03]$; planted kM38IX/kM38I per attempt, lower bound > 1; planted closure corruptions rejected. Every gate true (unknown = fail).

Figure 1: Design (from the frozen protocol and schedule; not a result). Source figures/design.tex, vector figures/design.pdf.

Instance. One GitHub `ubuntu-24.04` runner. Every process is a fresh process pinned to one CPU (`sched_setaffinity` before exec), with a clean environment. Every IC arm runs `ic_online_closure_indexlift_r770` with `KIC_INDEX_CANON=lift`, the full scan self-test, lift_scan target keys, Feistel, prefetch and the 16-byte compact table over the folded index.

Panel. 10 new public points, role `closureindexlift-rep-cold-v1`, seeds 0..9 (`make_targets.py`, `inputs/cold_targets.json`). The panel is disjoint from Q0, B's panel and every role the parent excluded, and from the parent's own `closureindexlift-cold-v1`. Each of these was regenerated and compared.

A role-agnostic scan (`check_panel_disjoint.py`, `inputs/panel_disjointness.json`) also looked for each of the 20 coordinates in three places:

- every file of `origin/main` under `experiments/` and `research/`;
- every file that any of the 858 other `origin/*` branches changes against `origin/main` (1,696,544 files);
- every file in the scratch worktrees `n83-blrp`, `n83-cli`, `n83-closure`, `n83-indexlift`, `n83-k899`, `n83-knownlog`, `n83-onlineframe` and `n83-rotkey` (30,084 files).

It found **0 hits**. As a positive control, the same `git grep` finds a parent panel x-coordinate in the parent's `cold_targets.json`, `schedule.json` and archived `runs.jsonl`. No scalar is known for any of the points.

Rank seeds. 1201-1230 are new on n = 83. The seeds used before are 7, 29, 101-102, 201-204, 301-309, 401-404, 501-504, 701, 801-816, 901-904 and 1101-1116. Primary pair j at point i uses seed $1200 + 3i + j$.

Sides and pairs.

side	processes, in order	role
A	kPI (rank stage, 600 attempts on K600, pair's seed, then the point), then cPI (B's verified table, the point)	index-lift row (baseline)
B	kM38I (rank stage, 16 attempts on M38, same seed, then the point), then cM38I (M38 verified table, the point)	closure + index lift (candidate)
Baa	kM38Iaa, then cM38Iaa	A/A copy of B

Each point gets one segment, Pi. Its four sub-blocks run in an order shuffled by `random.Random(2026102000+i)`, and the same generator draws each block's starting side:

- **primary:** k = **3 pairs** as 6 sides in strict alternation, ABABAB or BABABA. Pair j is sides $2j$-1 and $2j$. Both sides of a pair share the point and the rank seed.
- **A/A:** 2 pairs of B and Baa, alternating, with the seeds of primary pairs 1 and 2.
- **planted:** 1 pair of kM38I and kM38IX (the parent's `frame_plant`, about 1.96x per attempt), alternating, with the seed of primary pair 3.
- **rho:** one rT (tuned rho rT770), seed 1020000 + point seed.

The run starts with O1, which runs Q0 on cPI and cM38I as an exactness check. In total there are **232 processes**: **30 primary pairs** (30 kPI and 30 kM38I rank blocks, 30 cPI and 30 cM38I cold rows), 20 A/A pairs and 10 planted pairs. The primary starts with A at 7 points and with B at 3.

How E2E is formed (per side). E2E(side) = `timing_ms.rank_stage` of the side's rank process + `cold_in_process_ms` of the side's cold process. The cold process covers setup, index build and the one-target online phase. The rank stage covers the rank attempts and the linear algebra. Together that is the row's end-to-end cold definition. It is the parent's E2E, except that here the rank and cold terms come from one adjacent pair of processes with the same point and seed, not from arm-wide means.

Pairing basis. The Tier 1 paragraph says to "run both variants on the same pinned exposed CPU with the same workload and limits" and to "require repeated alternating pairs". Here a pair is the two variants on the same point with the same rank seed, adjacent in time on one CPU, in alternating order. That is also the unit of docs/IC_BENCHMARK_TIERS.md: the same point, alternating order, one allocation.

Primary (pre-registered).

- R = sum over the 30 primary pairs of E2E(A) / sum over the same pairs of E2E(B).
- 95% percentile interval from a **two-stage paired bootstrap**: draw the 10 points with replacement, then each drawn point's 3 pairs with replacement. A and B of a pair are always drawn together. 10,000 draws, `random.Random(20261020)`.
- **Improvement iff the lower bound > 1.** Regression iff the upper bound < 1. Otherwise not separated.
- **Invalid** if any gate is false, unknown or missing, if any control fails, or if any scheduled row is missing (time-guard skips included).

Controls.

- A/A: E2E(B) / E2E(Baa) and cold cM38I / cM38Iaa over the 20 A/A pairs. Both point estimates must lie in [0.97, 1.03].
- Planted timing: kM38IX / kM38I per attempt over the 10 planted pairs, with attempts paired. The lower 95% bound must exceed 1.
- Planted exactness: the three closure corruptions (a label, a duplicated orbit, a log) must be rejected on both closure bases before timing (`check_log_table.py`, verbatim).

Secondaries (descriptive; none changes the verdict).

- the geometric mean of the per-pair ratios;
- the paired rank-stage, cold, process-CPU-time and process-wall ratios;
- setup, index build, online and peak RSS;

- the per-point ratios, the ratio by pair position and the ratio by which side ran first;
- the probe law of kPI;
- S and the gap to rT.

Gates. These are the parent's, plus the Tier 1 records.

1. Before timing: B's table re-verified in the pinned oracle. Both closure headers regenerated to their frozen digests and passed by `verify_closure.py`, with the planted corruptions rejected. The closure tables checked, and the closure logs equal to B's.
2. Every IC process (`arm.py`): `KIC_INDEX_CANON=lift`, 0 roots checked against the selected canonicalizer, the full self-test, and the base's frozen index digests and **layout digests**. For K600 these are index-lift's `d8cfe443...` / `5f53764e...`.
3. Per point: one scalar across every IC process (both bases, cold and rank) and rT, **replayed in the oracle**. One relation and one probe count per point and base. One index and layout digest set per base, equal to the frozen one.
4. **Q0:** cPI must give B's archived identity (1,987,852 probes), and cM38I the closure protocol's frozen identity.
5. Rank processes: full rank and 0 failures. The written table must be **equal to the frozen verified table**. Every kM38I, kM38Iaa and kM38IX process with the same seed must have identical relation digests, probe vectors, root calls and counters.
6. **Probe-law band:** kM38I's pooled probes per relation over its 30 distinct seeds (480 attempts), divided by r / (166 E), must lie in [0.80, 1.20].
7. Tier 1 records:
   - every process's `Cpus_allowed_list` (read from `/proc` while it ran) is the execution CPU;
   - the CPU record (`scripts/host_cpu_record.py` from main: model, microcode, flags, caches, topology, kernel) is the same at the start and the end.

   Per run the receipt also holds:
   - process CPU time (user and system) and wall time;
   - producer peak RSS;
   - cgroup `cpu.stat` throttling, memory events, memory current and peak;
   - steal, IRQ and softirq on the execution CPU, and interrupts;
   - CPU pressure;
   - `/proc/schedstat` and `/proc/stat` scheduling counters;
   - voluntary and involuntary switches, and minor and major faults;
   - every raw failure (rows are never re-run, and failed rows are kept).

   The host probe and the allocation record identify the host, kernel and placement.
8. **Receipt integrity, fail closed** (main's hardened verifier plus schedule identity):
   - every raw record decodes, and its SHA-256 equals the wrapper's `raw_result_sha256`;
   - its arm, kind, panel, point and seed equal the schedule's;
   - each row's serial, segment, command line and stdout file are the schedule's, and rows ran in schedule order;
   - `runner_start.json` names this protocol's, this schedule's, `arm.py`'s and `run_schedule.py`'s digests and no smoke subset;
   - the binaries were unchanged at the end;
   - every required gate must be present and `true`.

Power and runtime

`power_check.py` simulates receipts of exactly this design and applies the frozen statistic and bootstrap (`analysis/power.json`; 400 simulations per cell, 500 draws each). Its prior is the parent's archived run 39335956567 (`make_power_prior.py`, `inputs/power_prior.json`, with every raw record's digest checked).

- kPI is resampled from its 4 archived blocks.
- kM38I is drawn from the 256 archived attempts.
- The cold times come from the 20 archived points.
- Each process carries a noise factor drawn from the archived A/A ratios. This overstates the per-process noise by about sqrt(2), so the power figures are conservative.

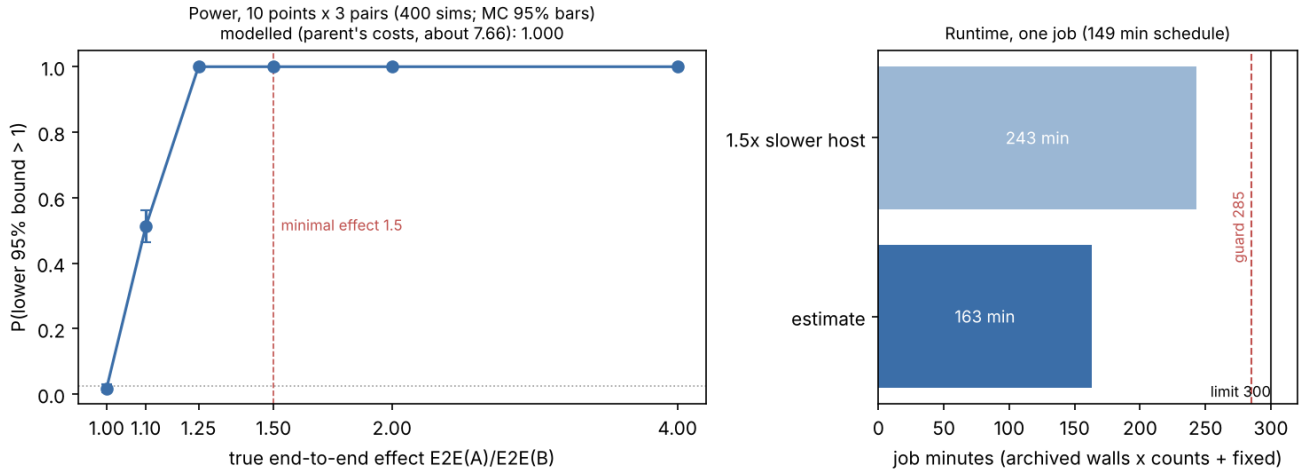


Figure 2: Design-stage power and runtime (not a result), from `analysis/power.json` by `make_figures.py` (svg, pdf).

true end-to-end effect	P(lower bound > 1)
1.00 (no gain; type I)	0.018
1.10	0.51
1.25	1.00
1.50 (minimal effect of interest)	**1.00**
2.00	1.00
4.00	1.00
modelled: the parent's costs, about 7.66	**1.00** (median R 7.68, median interval hi/lo 1.06)

The A/A point estimates fall inside [0.97, 1.03] in 100% of simulations. The probe-law gate passes at the archived law in 100% of simulations and rejects a law 1.5x worse in 100% of them (480 attempts). The planted lower bound exceeds 1 in 100% of simulations.

Runtime. The estimate uses the parent's mean process walls: kPI 162.6 s, kM38I 22.9 s, cold 18.9 s, kM38IX 28.4 s and rT 3.1 s. That gives a P segment of about 891 s and a schedule of about **149 min**. Adding the parent's fixed cost before the schedule (11.7 min: restore, build, tests, table checks) and after it (about 2 min) gives a job of about **163 min (2.7 h)**, or about **243 min** on a host 1.5x slower.

The time guard is at 285 min of job time and the job limit is 300 min, so **one job fits**, and the run is not split. The guard skips a segment if the elapsed time plus the larger of 1,300 s and 1.1x the longest P segment so far would pass 285 min. Any skip makes the run invalid. That outcome is recorded, never patched.

Exactness checks done before freezing (cloud-agent VM, correctness only)

- `restore.py--crypto-repo/home/user/crypto` passed. It checked the r770 snapshot against the reconcile protocol's digests, the derivation chain, both closure headers, the parent protocol and the 25 copied files.
- `check_log_table.py` passed for K600, M38 and M10, and the planted corruptions were rejected (`smoke/log-table-check.json`).
- 23 verifier unit tests passed (`test_verify_receipt.py`). They cover:
  - a missing, unknown or extra gate, each of which is invalid;
  - raw tampering and identity mismatch;
  - command and runner identity;
  - a paired bootstrap that keeps the pairs together;
  - strict alternation and seed sharing in the frozen schedule.
- A smoke of O1 and the first process of every arm in P5 gave 10 of 10 valid rows (`smoke/README.md`, `smoke/correctness_smoke.json`):

- Q0 equal to the archived and frozen identities;
- one scalar per point, replayed;
- the index and layout digests equal to the frozen ones on K600 and M38;
- the rank tables equal to the frozen tables;
- kM38I and kM38Iaa identical on seed 1213;
- placement and CPU identity recorded and passing.

The probe-law band is not evaluable on 2 blocks. The protocol was frozen before the smoke and not changed after it.

Trigger

The run is the workflow `n83-closure-indexlift-rep.yml`, dispatched with `workflow_dispatch` on ref `claude/n83-closure-indexlift-rep`. If dispatch is refused for the integration, a push trigger is added. It is restricted to this branch and to the experiment's non-receipt paths, as in `n83-closure-indexlift.yml`. The run URL is recorded in the result once the receipt is archived.

Not included

- No result. When the run finishes, its receipt is archived under `run_github_<id>/` and analysed by the frozen `verify_receipt.py`. Any benchmark copy follows AGENTS.md "Performance results also land in `benchmarks/`".
- The M10 secondary is not timed. Its base is still verified before timing, because `check_log_table.py` is verbatim.
- Factor-base construction is charged to no arm, as in every n = 83 row.

Files

File	Role
`protocol.json`, `make_protocol.py`, `schedule.json`, `make_schedule.py`	frozen protocol, its builder, frozen run order
`restore.py`	restore crypto 770f6a69, derive the producer, regenerate the closure headers, check every parent digest and every local pin
`make_online_source.py`, `make_*_source.py`, `online/`, `lift2/`, `make_closure_base.py`, `verify_closure.py`, `arm.py`, `check_log_table.py`, `inputs/factor_logs_*`, `inputs/parent_closure_protocol.json`, `inputs/local_*.json`, `inputs/archived_identities.json`	the parent's files, byte for byte
`run_schedule.py`	the parent's runner plus the CPU record, placement and scheduling counters
`make_targets.py`, `inputs/cold_targets.json`, `check_panel_disjoint.py`, `inputs/panel_disjointness.json`	the new panel and its disjointness record
`make_power_prior.py`, `inputs/power_prior.json`, `power_check.py`, `analysis/power.json`	power prior (run 39335956567), simulation, result
`verify_receipt.py`, `test_verify_receipt.py`	gates, statistics, fail-closed decision rule; its tests
`figures/design.*`, `figures/power.*`, `make_figures.py`	design diagram (TikZ source, PDF, PNG); power and runtime figure (SVG, PDF, PNG)
`smoke/`	local correctness smoke
`README.pdf`	this report: `pandocREADME.md-oREADME.pdf--pdf-engine=pdflatex-Hreport_header.tex--lua-filterreport_filter.lua-Vgeometry:margin=2cm-Vfontsize=10pt-Vcolorlinks=true`